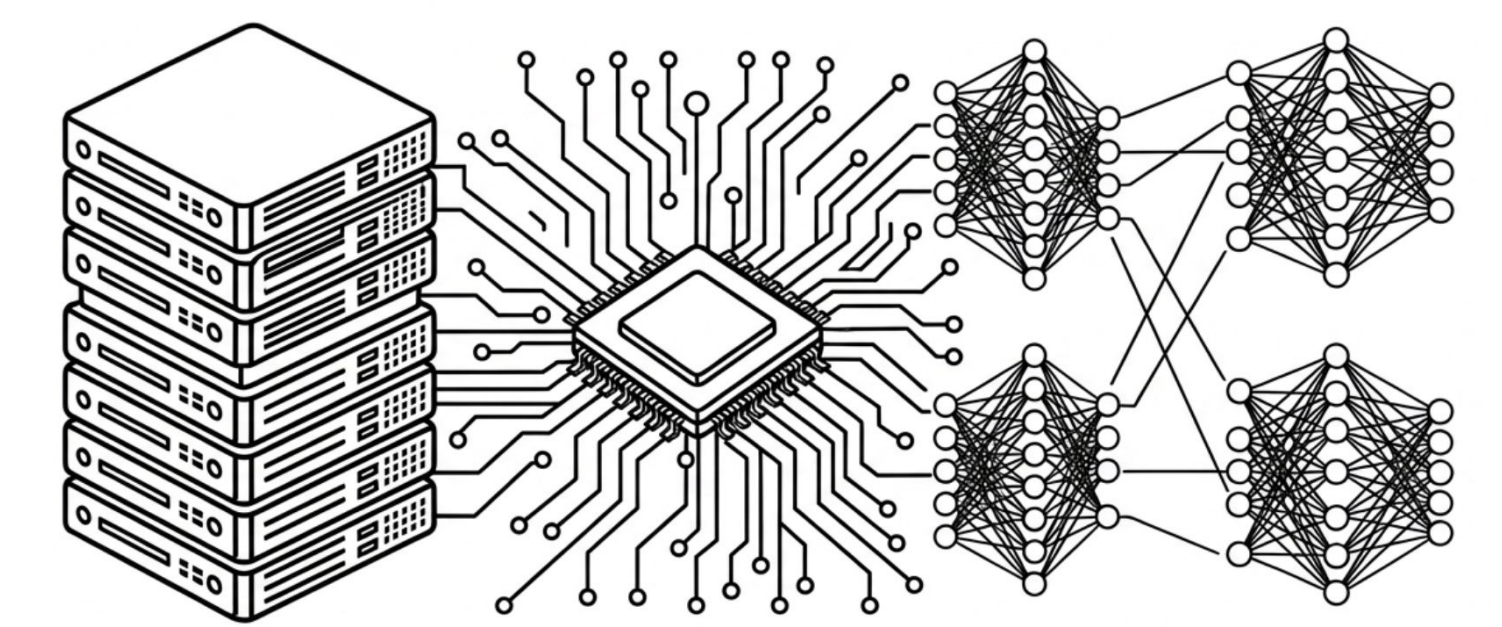
modelli a manetta con vLLM

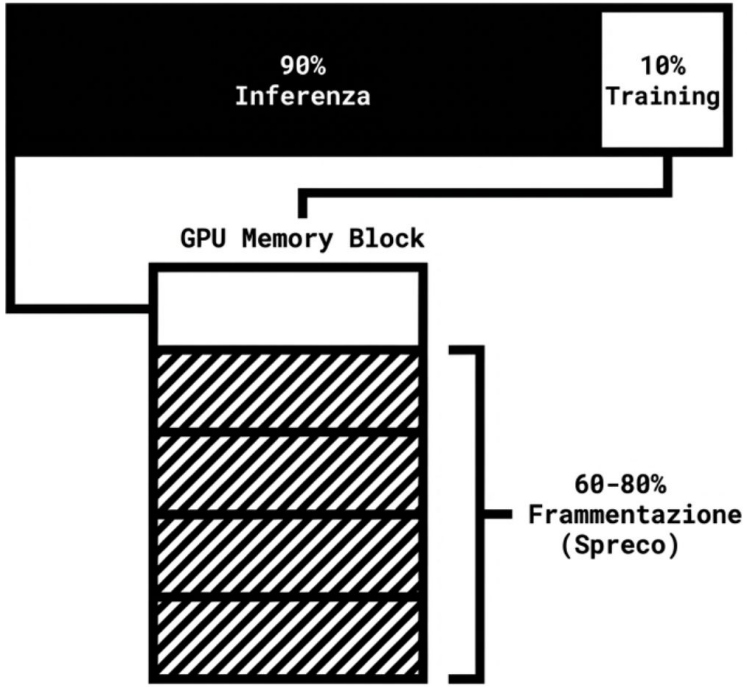
> root@linux-meeting-palermo:~# ./start_presentation.sh --topic 'vLLM e llm-d'

Fabrizio Soppelsa = (Solution Architect @ Red Hat) & (studente IA @ UNIPA)
Linux Meeting Palermo 28/03/2026



il vero costo degli LLM? l'inferenza

- ➔ 90% costi operativi di modelli di linguaggio: non training, ma da **inferenza**
- ➔ vero collo di bottiglia non calcolo, ma **memoria**
- ➔ problema: spreco di risorse costose come GPU e VRAM



agenda

- training e inferenza
- vLLM
  - come fa ad andare così veloce
  - naive hf vs vLLM
- llm-d
  - inferenza distribuita
  - un simulatore
- conclusioni

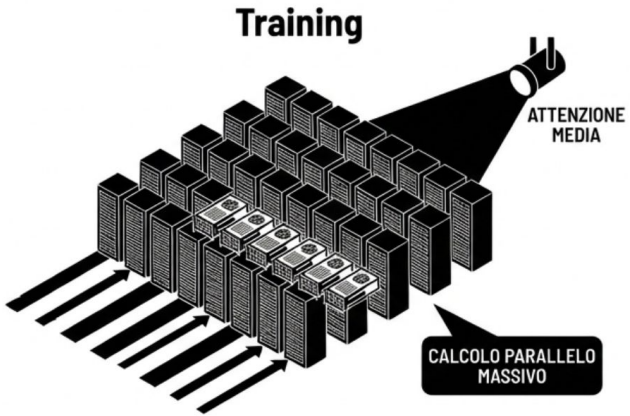
training e inferenza

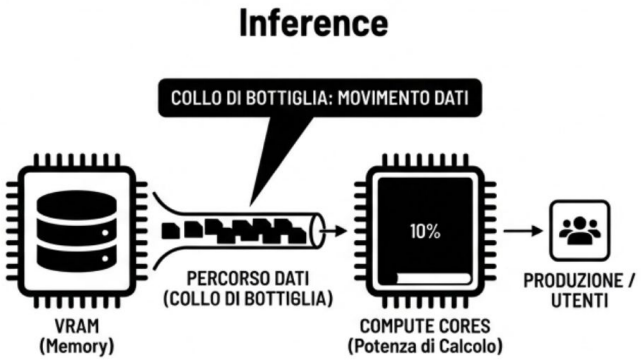
ciclo di vita di un {L,S,V}LM

15%	15%	70%
Training	**Fine-tuning**	**Inference (Produzione)**
• **Settimane** o mesi di calcolo. • **Milioni di dollari** di costo. • Si fa una volta sola. • Richiede cluster di migliaia di GPU. • Il modello impara i pesi.	• **Ore o giorni** di calcolo. • Si fa **poche volte.** • Adatta il modello base a un task specifico (LoRA, QLoRA). • Opzionale.	• **Millisecondi** per richiesta. • **Si fa milioni di volte al giorno.** • È qui che si spendono i soldi. • Il modello genera token uno alla volta, autoregressivamente. • Il 90%+ del costo totale del ciclo di vita risiede qui.

inferenza

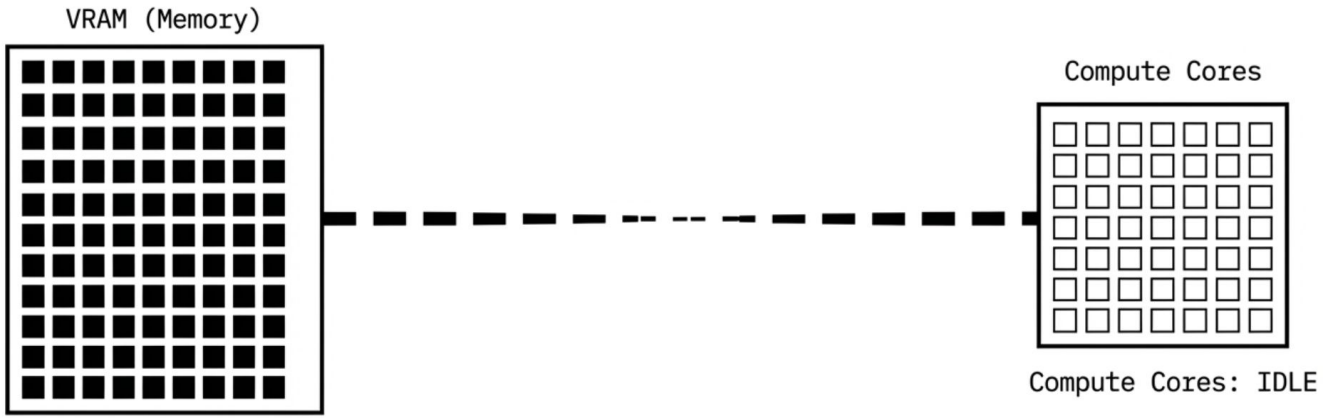
training cattura attenzione dei media ma inferenza è dove i sistemi collassano. servire un LLM su larga scala non è un problema di AI ma di **infrastruttura**, e non calcolo, ma capacità di **spostare dati** in real time



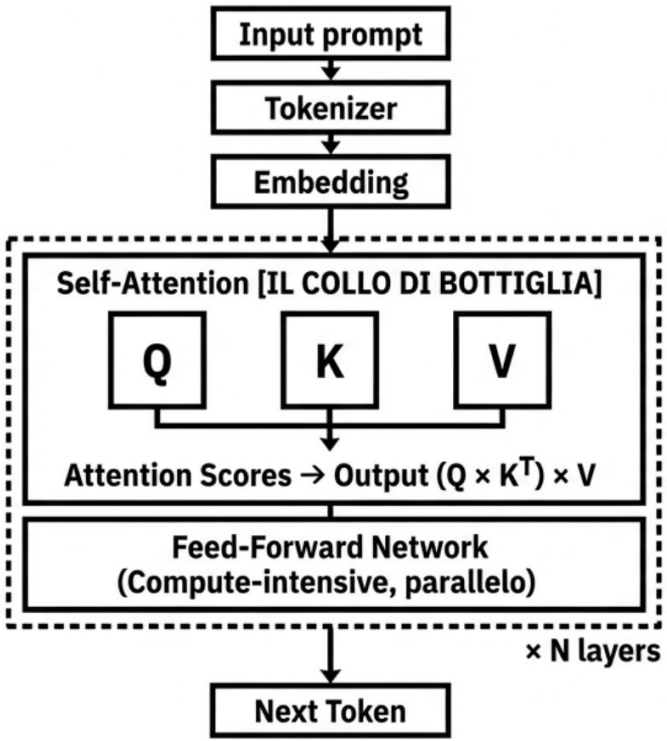


larghezza di banda, non teraflops

generazione autoregressiva (decoding) processo memory-bound. per ogni singolo token, GPU carica vari GB di pesi del modello da VRAM ai core. capacità di compute rimane **inutilizzata** mentre i core aspettano



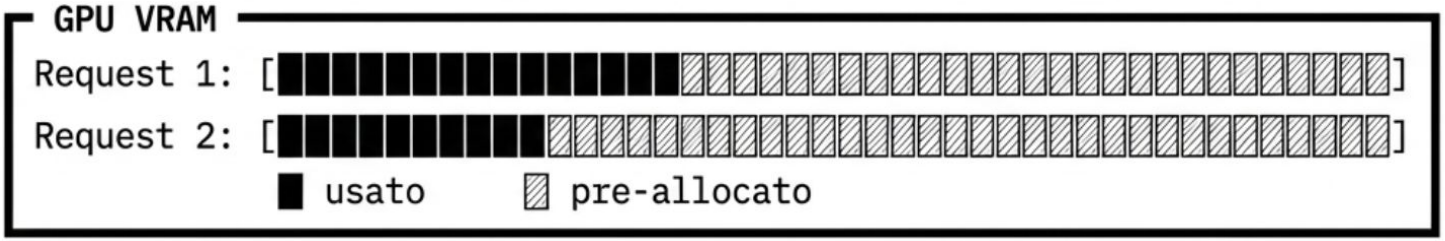
la KV cache, croce e delizia



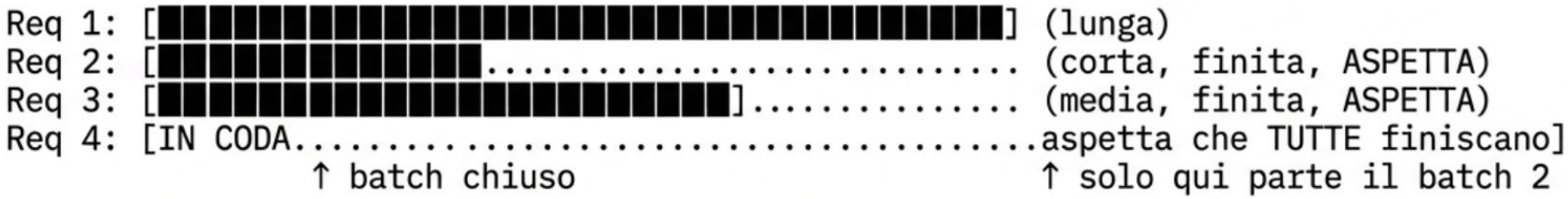
- ➔ ogni token: modello ricalcola attenzione con tutti i token precedenti (che non cambiano)
- ➔ li **salva** in KV cache
- ➔ problema: la KV cache satura la VRAM crescendo linearmente a ogni token

In Llama-8B con context 8K una singola richiesta occupa 2GB di memoria GPU solo per la KV cache

KV cache: problema 1 e problema 2



SPRECO: 60-80% vuoto



problema 1: lunghezza risposta sconosciuta, allocazione contigua, spreco di **spazio**

problema 2: il batch non si libera finché ultima richiesta non finisce, **bloccando** altre richieste

vLLM

alla ricerca del SOTA nel throughput

- ➔ vLLM (2023)
- ➔ trasforma l'inferenza su GPU in un **problema di sistemi operativi**
- ➔ PagedAttention
- ➔ continuous batching
- ➔ speculative decoding etc.
- ➔ meta, mistral, IBM, etc.

[> PAPER: Efficient Memory Management for Large Language Model Serving with PagedAttention, Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, UC Berkeley, 2023]

vLLM Engine

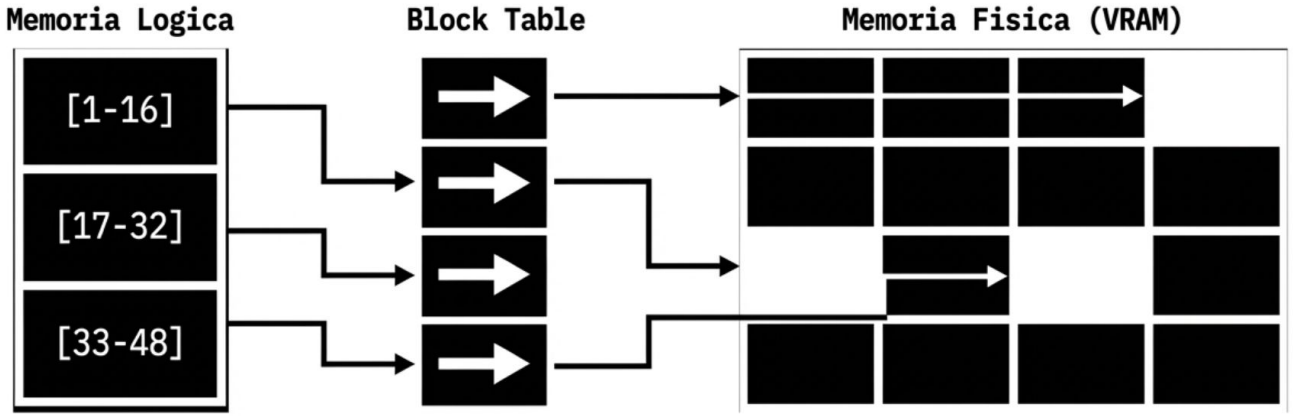
Sviluppato a UC Berkeley

High-throughput & Memory-efficient

Integrazione nativa Hugging Face

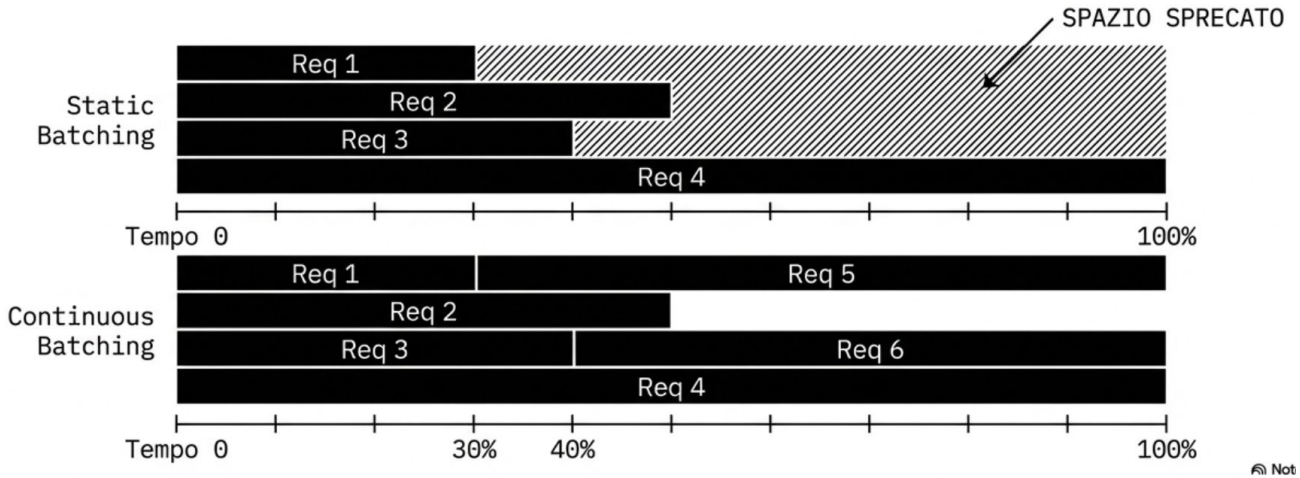
PagedAttention

divide la KV cache in pagine di dimensione fisse (es. 16 token). memoria allocata on-demand. quando sequenza cresce, viene mappata nuova pagina fisica, anche non continua. la block table mantiene i puntatori.



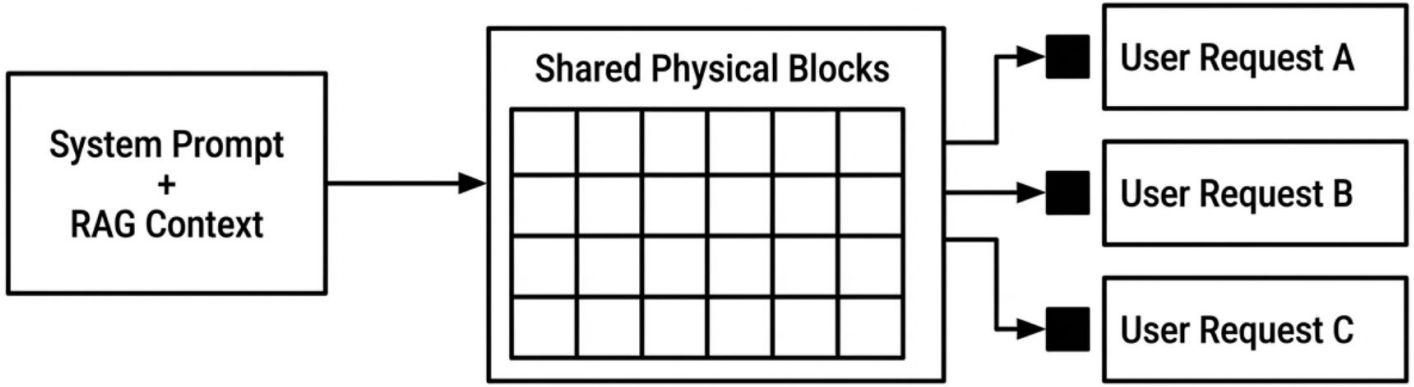
continuous batching

aggiorna il batch a ogni token: appena una sequenza termina/libera spazio, suo slot viene subito assegnato a una nuova richiesta, mantenendo la GPU quasi sempre occupata anche con sequenze di lunghezza diversa



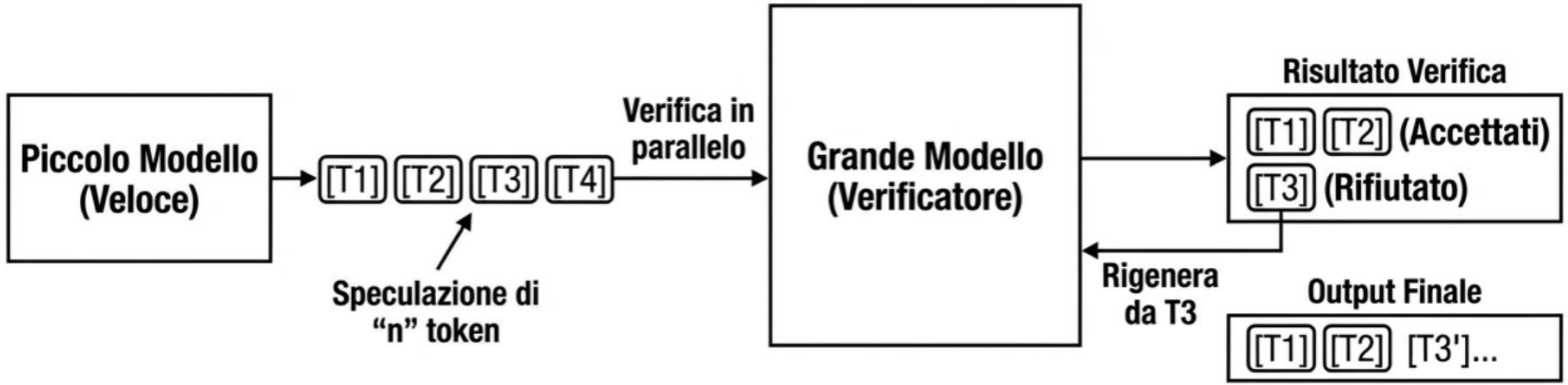
prefix caching

per chat, documenti fissi o RAG, il modello ricalcola solo il nuovo testo e riattacca la generazione alla fine del prefisso già in cache, riducendo latenza e costo computazionale



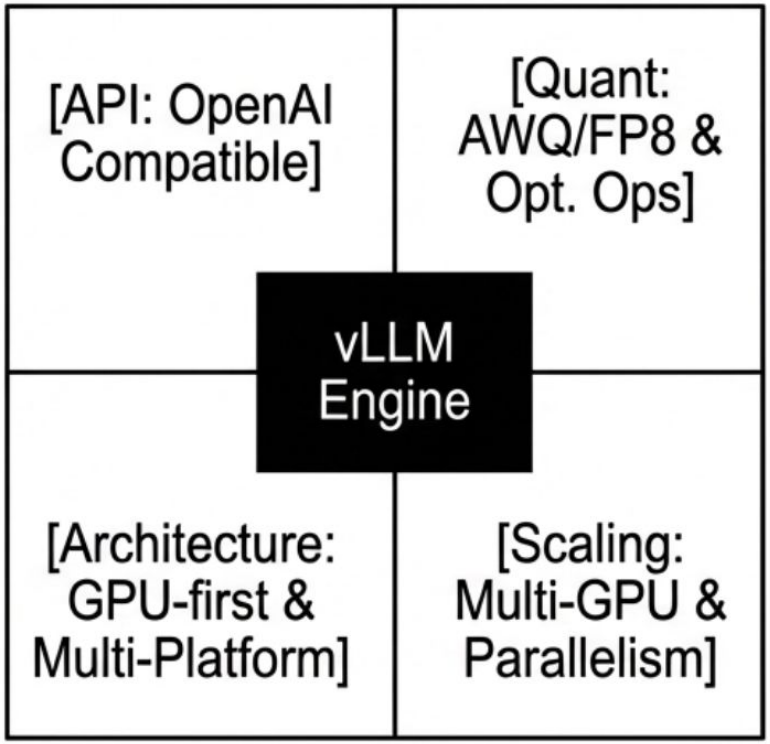
speculative decoding

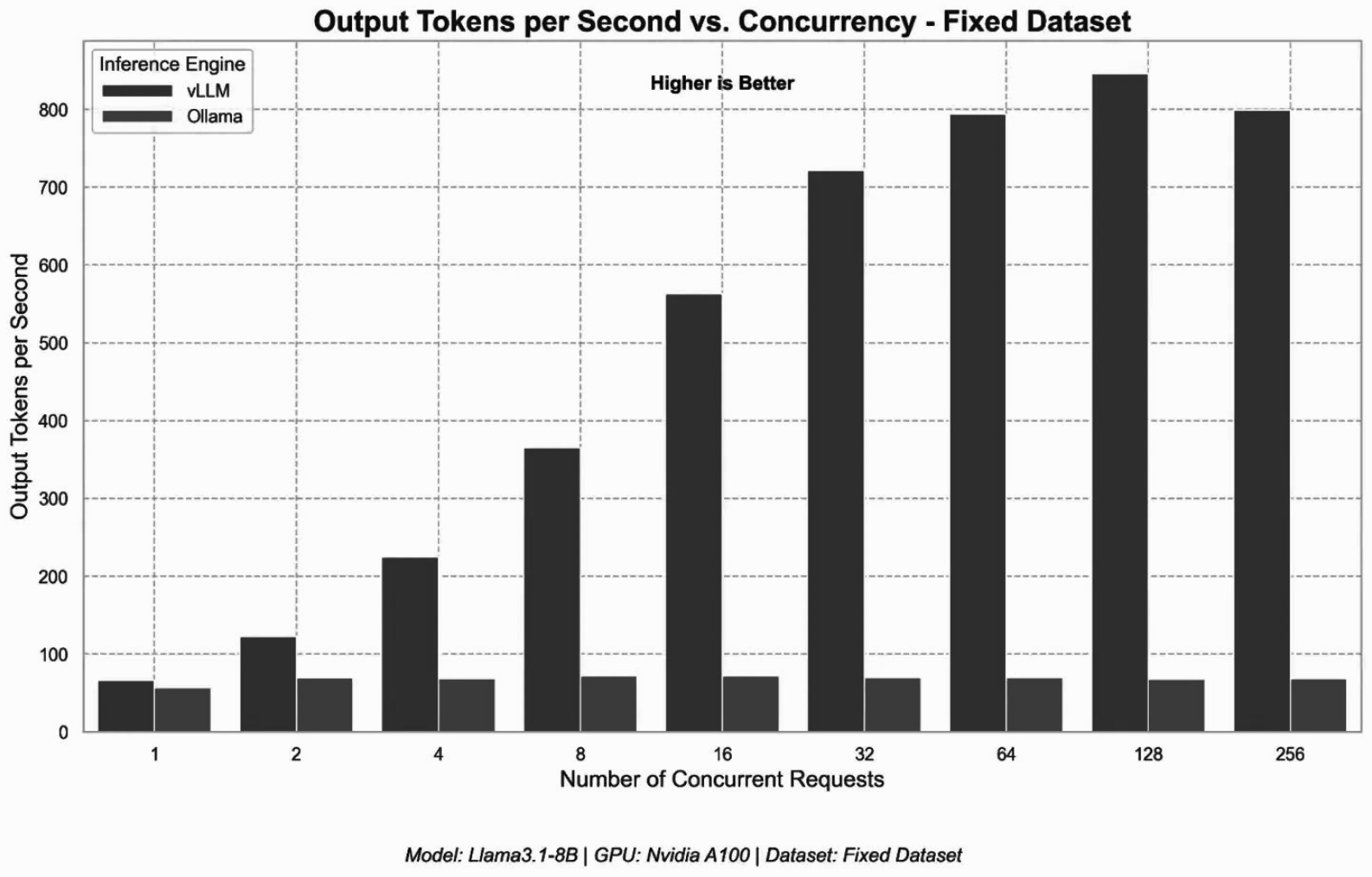
usa un modello più piccolo per "speculare" *n* token in anticipo; il modello grande li verifica in un solo passo e mantiene solo i token accettati, eventualmente li rigenera, genera i token più difficili

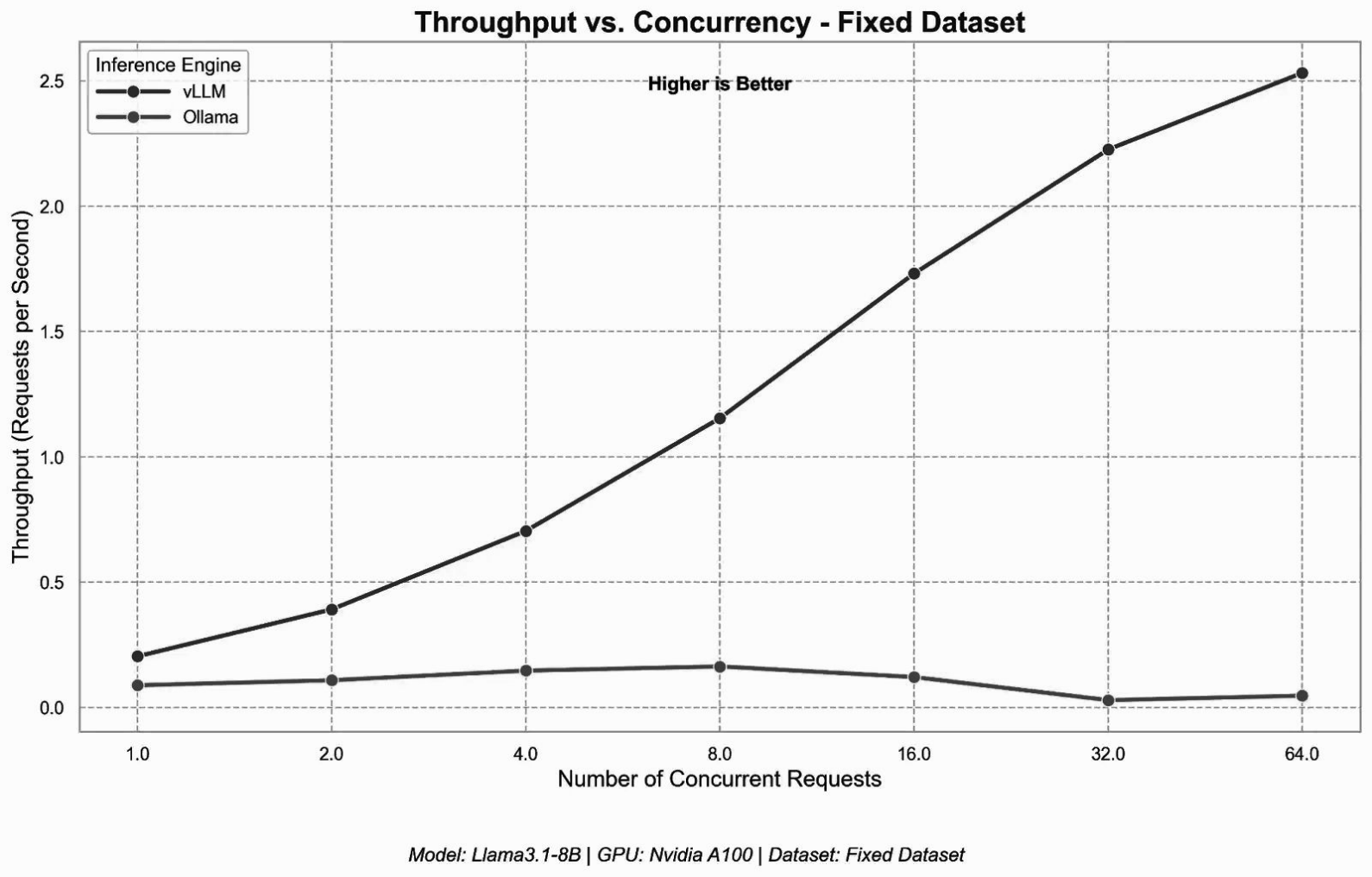


misc features

- ➔ GPU-first: riduce al minimo uso di CPU
- ➔ integrazione avanzata con quantizzazione e operatori ottimizzati (GPTQ, AWQ, GGUF…)
- ➔ API standard compatibili con OpenAI
- ➔ scaling e parallelismo nativi anche multi-GPU

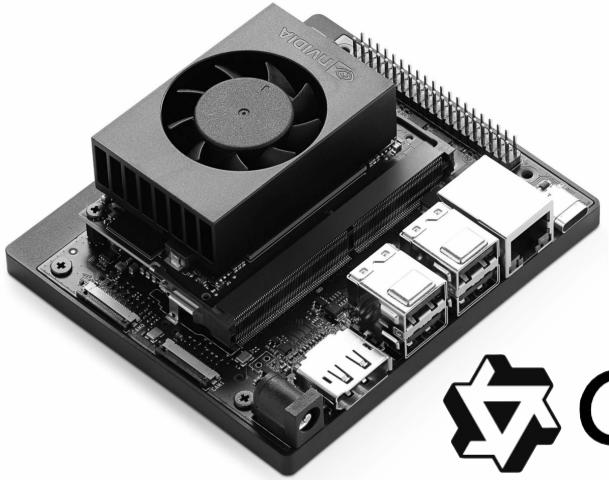



Output Tokens per Second vs. Concurrency - Fixed Dataset
Inference Engine
vLLM
Ollama
Higher is Better
Output Tokens per Second
0
100
200
300
400
500
600
700
800
1
2
4
8
16
32
64
128
256
Number of Concurrent Requests
Model: Llama3.1-8B | GPU: Nvidia A100 | Dataset: Fixed Dataset


Throughput vs. Concurrency - Fixed Dataset
Inference Engine
vLLM
Ollama
Higher is Better
Throughput (Requests per Second)
2.5
2.0
1.5
1.0
0.5
0.0
1.0
2.0
4.0
8.0
16.0
32.0
64.0
Number of Concurrent Requests
Model: Llama3.1-8B | GPU: Nvidia A100 | Dataset: Fixed Dataset

NVIDIA

Jetson Orin Nano
Developer Kit



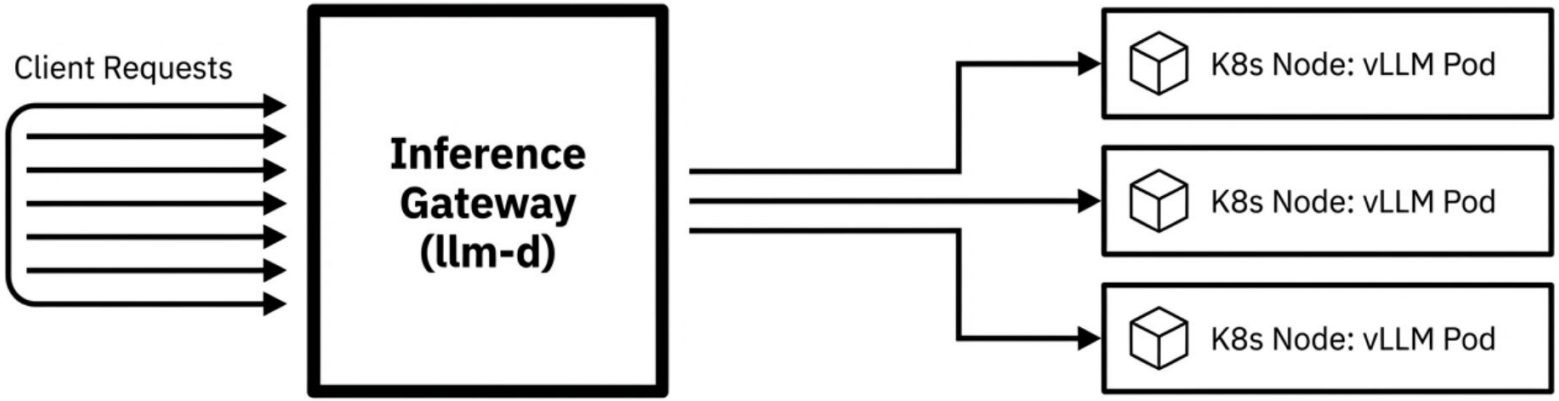
Qwen

vLLM demo

llm-d

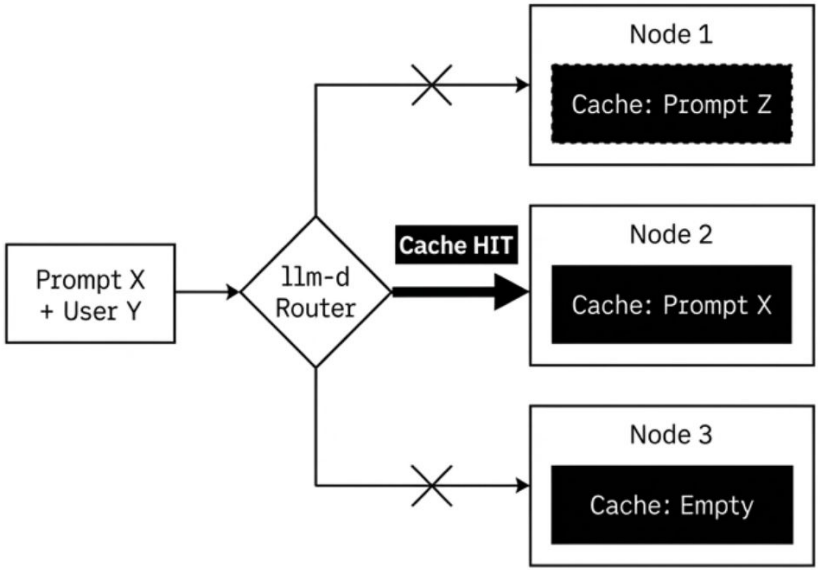
llm-d scala l'architettura di vLLM su K8s

un singolo nodo, per quanto ottimizzato, è limitato.
per scalare a livello enterprise serve orchestrazione.
llm-d trasforma vLLM in un inference server
distribuito con architettura K8s-native



KV-cache-aware routing

- ➔ intelligenza di routing
- ➔ il router conosce lo stato della cache di ogni nodo
- ➔ es. se nodo ha già elaborato in contesto RAG, llm-d instrada nuove richieste su quel nodo specifico



NVIDIA

Jetson Orin Nano
Developer Kit

llm-d simulator

conclusioni

- ➔ vLLM 0.14.1+ standard de facto per servire language model su linux in produzione: >790tps
- ➔ nel 2026 vLLM ha aggiunto supporto a NVIDIA Nemotron-3 Super, modelli multimodali e ha potenziato speculative decoding +50%
- ➔ vLLM gira nativamente su Linux metal, K8s, cluster multi GPU con ray, API OpenAI…

grazzi assai 🎉

```
> exit
> Connection to cluster closed
```